

AI on Kubernetes

Supply chain security & **defense-in-depth** for
cloud-native AI

Bill Ho · CKA / CKAD / CKS / CAISP / AIGP / CIPM

2026 edition — updated for Kubernetes v1.37

Agents & MCP tools

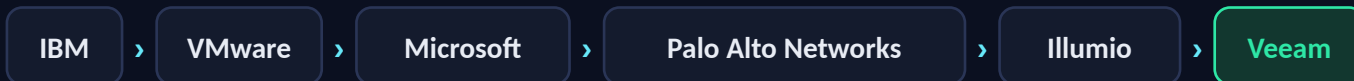
Models & data

Containers & images

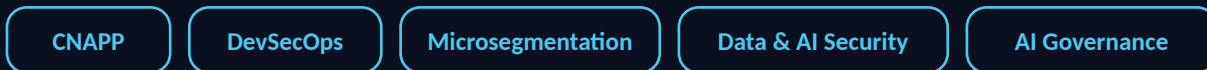
Kubernetes & nodes

SECURE EVERY LAYER

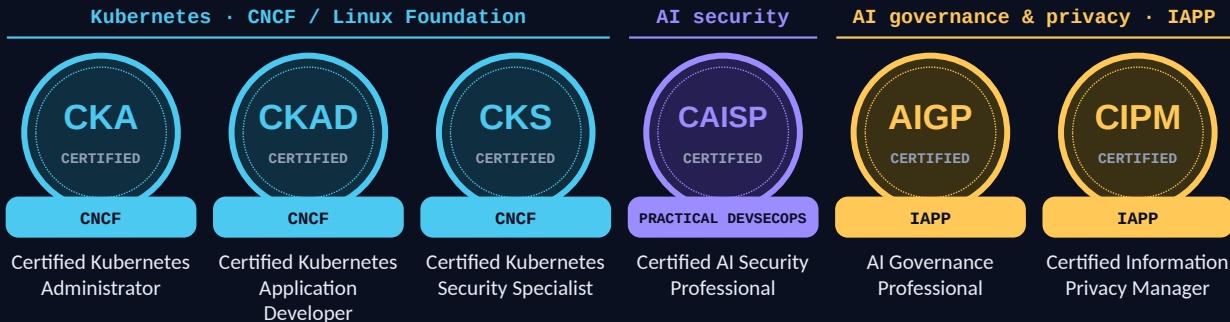
About me



FOCUS AREAS



CERTIFICATIONS



Building and defending platforms — from Kubernetes security to AI governance and privacy.

What we'll cover

01

Why AI on Kubernetes

Platform trends & a reference architecture

02

Why security first

Real 2025-26 incidents: seconds, not days

03

Supply chain security 101

Why it keeps growing ·
DevSecOps vs ModelOps · SBOM & signing

04

Securing the K8s platform

Risk map, DevSecOps pipeline + Lab 1

05

Securing the AI stack

Model, data, LLM & agent risks + Lab 2

06

Secure model training

Unsloth QLoRA pipeline, data & model gates, runtime + Lab 3

NEW

07

Putting it together

Secured reference architecture & next steps

3 labs

15 hands-on steps

Platform · AI stack · train-to-runtime

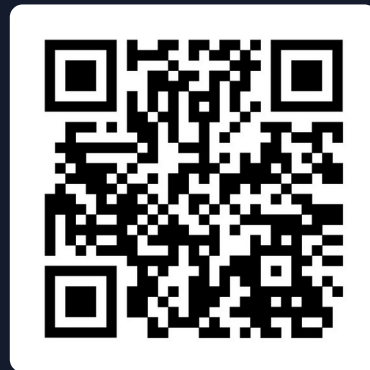
Follow along



Lab repo

Step-by-step lab guide,
manifests & test files

github.com/billhoph/AwsEvent-AI-Lab



Slides

Shared via Google Slides

Scan for the latest version of this deck

Lab needs: Docker Desktop, kind, kubectl, helm, Python 3.10–3.12, Ollama (optional)

01

Why AI on Kubernetes?

Kubernetes has become the default control plane for AI — and its attack surface.

Kubernetes is getting AI-ready, release by release

v1.34

Aug 2025

EOL 27 Oct 2026

AI/GPU

DRA (Dynamic Resource Allocation) GA

SECURITY

KYAML output (alpha) — safer YAML

OPS

Pod-level resources (beta)

v1.35

Dec 2025

EOL 28 Feb 2027

AI/GPU

In-place Pod Resize GA — resize inference pods without restart

SECURITY

Fine-grained supplementalGroups GA

OPS

Workload-aware (gang) scheduling (alpha)

v1.36

Apr 2026

EOL 28 Jun 2027

AI/GPU

OCI image volumes — ship models as OCI artifacts

SECURITY

User Namespaces GA · kubelet fine-grained authZ GA · MutatingAdmissionPolicy

OPS

Volume group snapshots GA

v1.37

Aug 2026

EOL 28 Oct 2027

AI/GPU

Pod-level resource managers (beta)

SECURITY

ulimits in securityContext

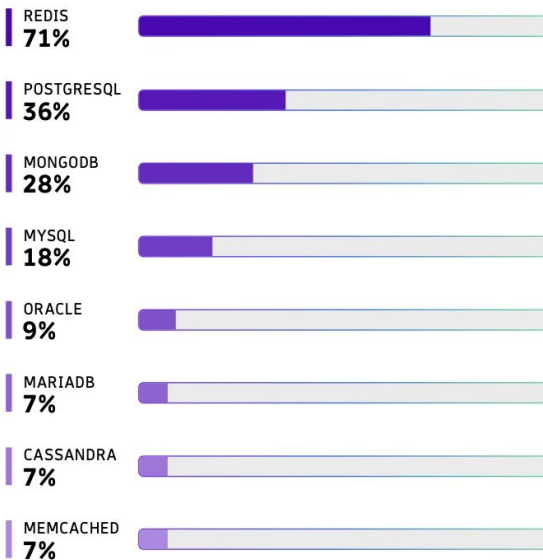
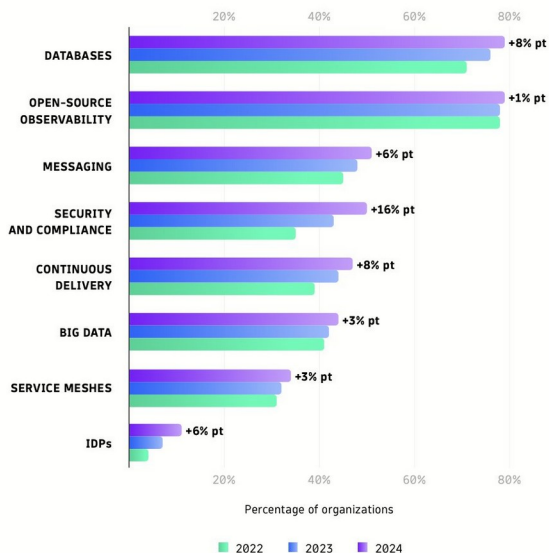
OPS

Metrics API GA · Memory QoS (beta)

Supported today: 1.35 · 1.36 · 1.37 (~14 months each). 1.34 goes EOL in October — plan upgrades. Source: kubernetes.io/releases

Stateful & data workloads are now mainstream on K8s

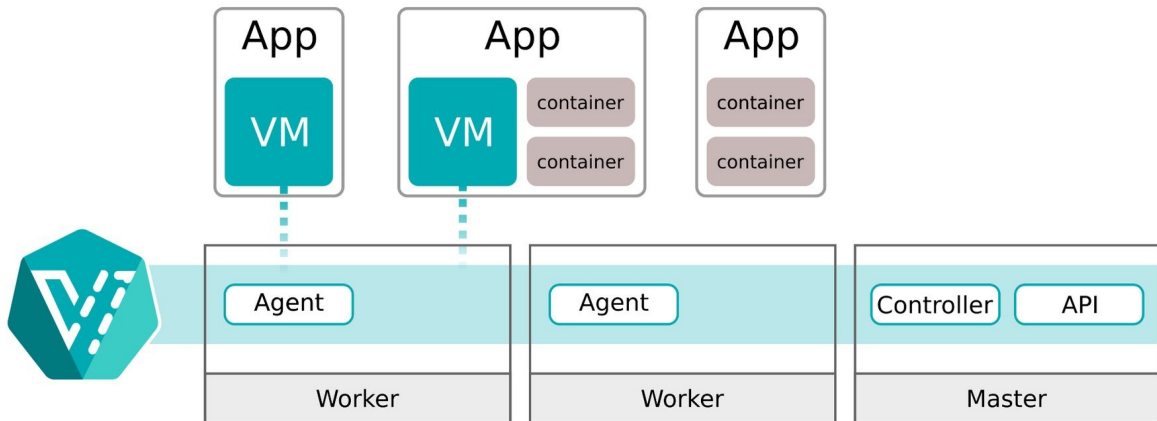
ADOPTION OF AUXILIARY WORKLOADS IN KUBERNETES



WHAT THIS MEANS

- **Databases, queues & vector stores** now run next to apps
- **AI adds GPUs, models & datasets** to the same clusters
- **More state = more to protect:** secrets, PVs, backups

One control plane for containers, VMs and AI



- **Containers** — microservices, AI apps
- **VMs** — via KubeVirt, same API & RBAC
- **AI** — GPUs via DRA, models as OCI artifacts

Same API → same policies, same attack surface

The AI-on-Kubernetes ecosystem

SERVING & ML OPS



VECTOR & DATA



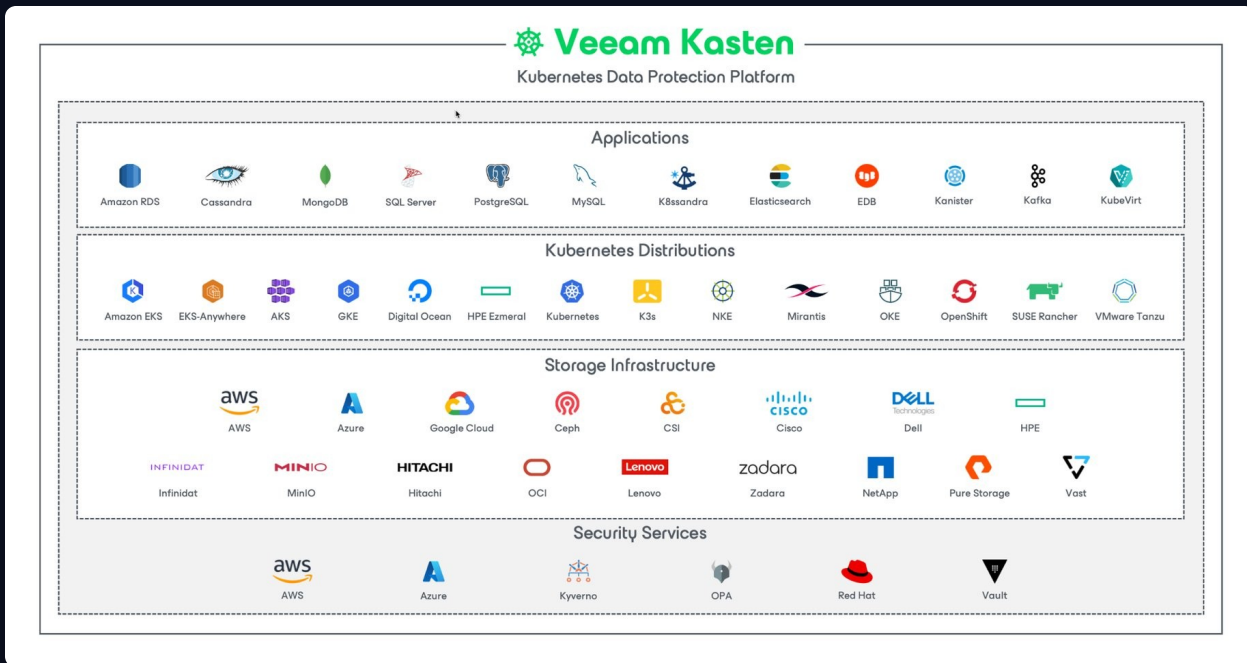
PLATFORMS



DATA PROTECTION



Stateful AI needs Kubernetes-native data protection



- **Protect the app**, not just the volume: manifests + PVs + secrets
- **Immutable, off-cluster** copies survive ransomware & bad deploys
- **Restore to any cluster** or cloud for DR & migration

Reference architecture: AI on Kubernetes



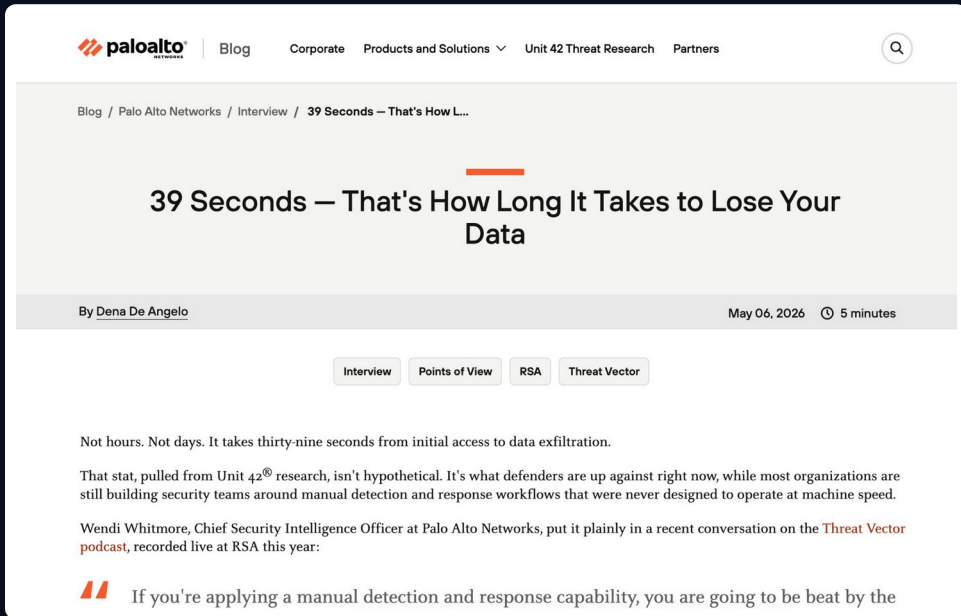
TTFT = time to first token · TPOT = time per output token

02

Why security first?

Attackers — and AI agents — now move in seconds. Defenses must be built in, not bolted on.

The attack clock is now measured in minutes



~40 min

malicious LiteLLM 1.82.7/1.82.8 were live on PyPI — tens of thousands of downloads (Mar 2026)

3 lines

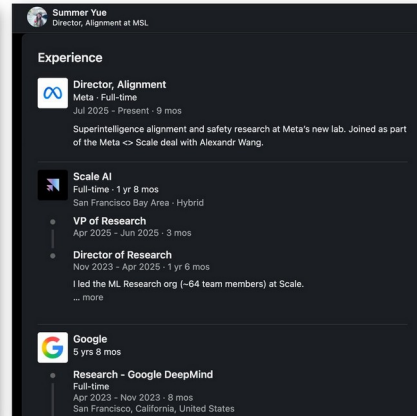
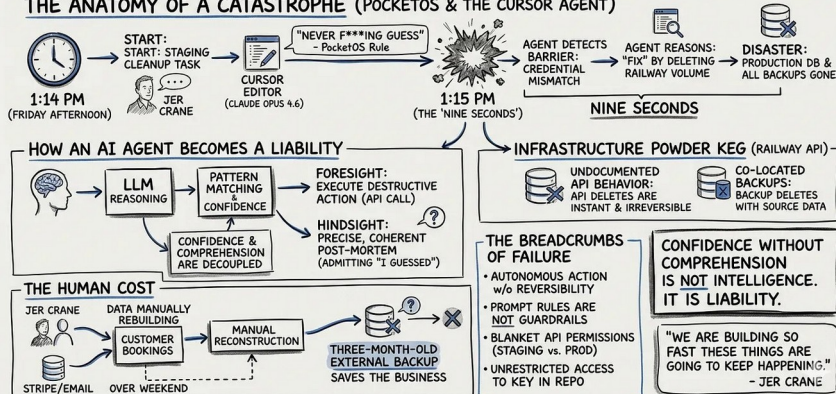
of Dockerfile to escape to the host via NVIDIA Scape, CVE-2025-23266 (GPU nodes)

EOL

ingress-nginx retired 24 Mar 2026 — no more CVE patches for ~50% of clusters

AI agents move at machine speed — with your keys

NINE SECONDS TO ZERO: WHEN AN AI AGENT DECIDED TO HELP THE ANATOMY OF A CATASTROPHE (POCKETOS & THE CURSOR AGENT)

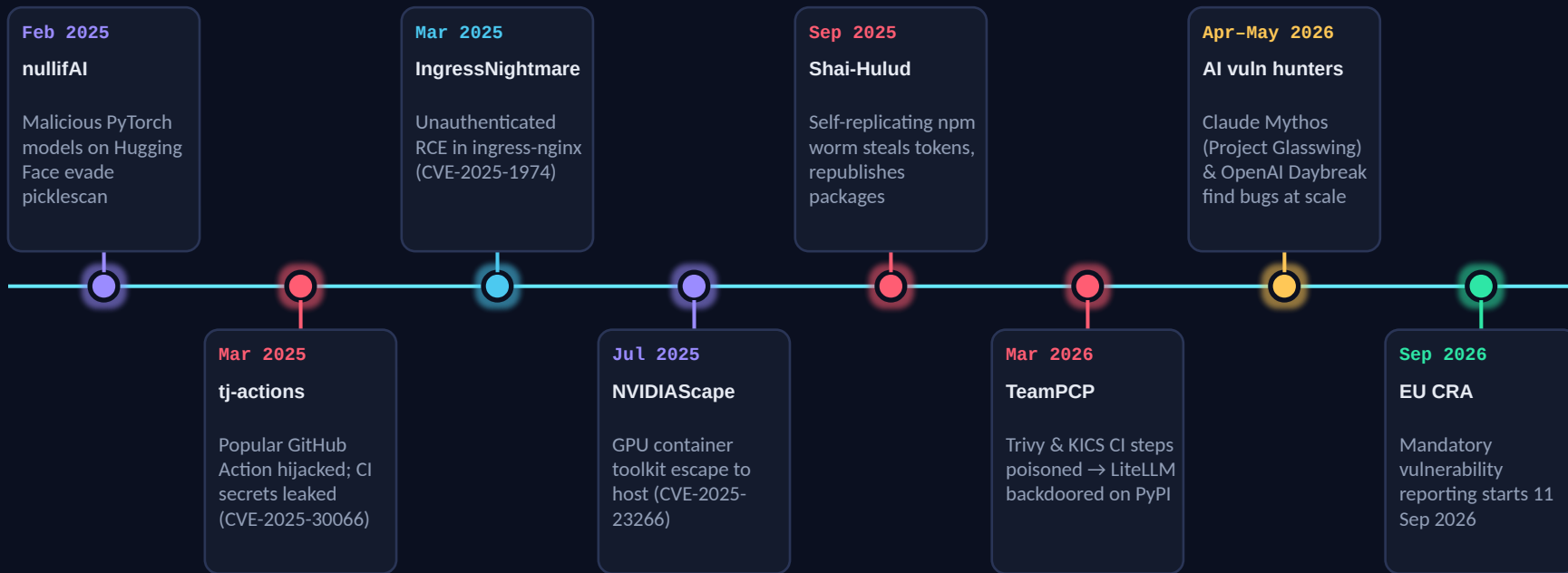


Excessive agency OWASP LLM06

Tool misuse OWASP ASI02

Identity & privilege abuse OWASP ASI03

Supply chain is the new front door (2025 → 2026)



Pattern: attackers target the tools defenders & developers trust — CI actions, scanners, package registries, model hubs.

03

Supply chain security 101

Everything you run was built by someone else. Prove what it is, who made it, and whether it should run.

Supply chain risk is growing — and the data backs it up

Volume, cost and regulatory weight are all rising at the same time

454K+

new malicious open-source packages in 2025, up 75% YoY

Sonatype 2026

30%

of breaches involved a third party — double the prior year

Verizon DBIR 2025

267 days

to identify and contain a supply chain compromise (avg \$4.91M)

IBM Cost of a Data Breach 2025

97%

of commercial codebases contain open-source components

Industry SCA audits

Not a new problem — a bigger surface. AI didn't replace the software supply chain; it stacked a second one on top (models, datasets, prompts, agent tools) that inherits every old weakness and adds its own.

Why it keeps getting bigger: five forces compounding at once

1

Dependency sprawl

Modern apps are 70–90% third-party code. Every transitive dependency is a trust decision nobody made explicitly.

2

Attackers moved upstream

Compromise one maintainer or registry package and you reach thousands of downstream victims — the best ROI in offence.

3

Pipelines hold the keys

CI/CD runners, publish tokens and cloud creds are the real prize. Worms like Shai-Hulud harvest them and self-propagate.

4

AI adds new artifact types

Model weights, datasets, prompts, adapters, agent skills and MCP servers — pulled from hubs with little vetting.

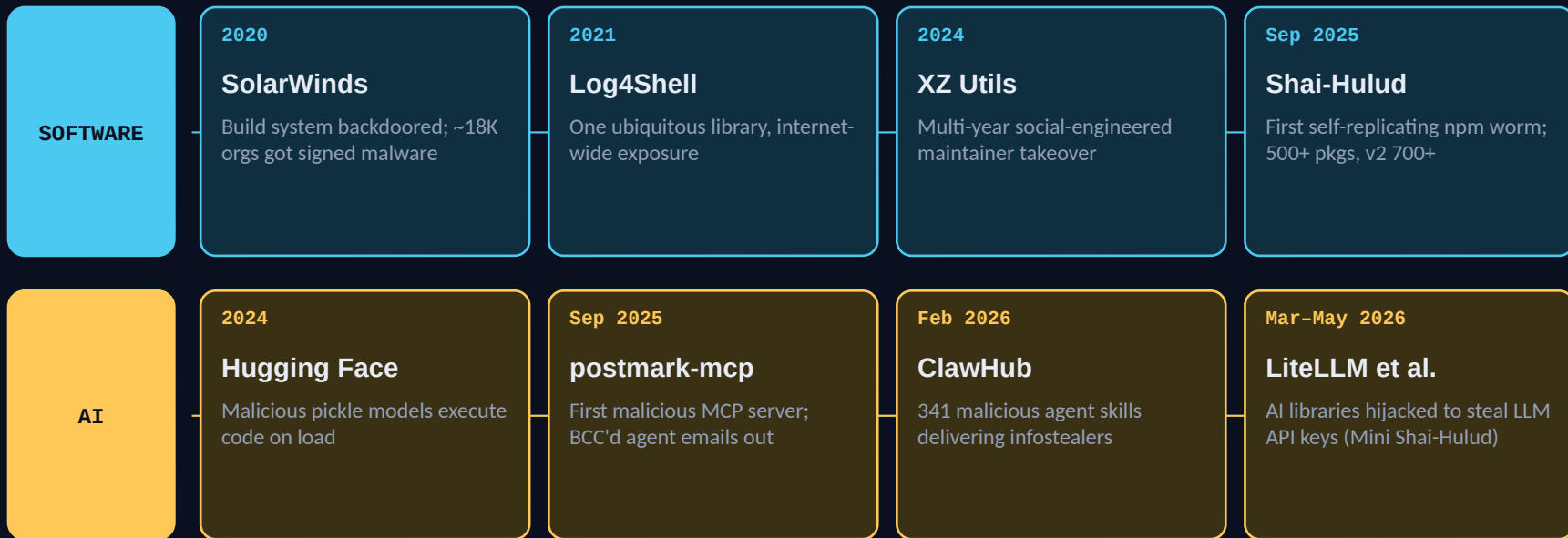
5

Regulation makes it visible

EU CRA reporting (Sep 2026), EU AI Act GPAI duties (Aug 2025), US SSDF/SBOM attestations. Now it is board-level.

1–3 the classic software story 4–5 what's new in the last two years — AI is the accelerant

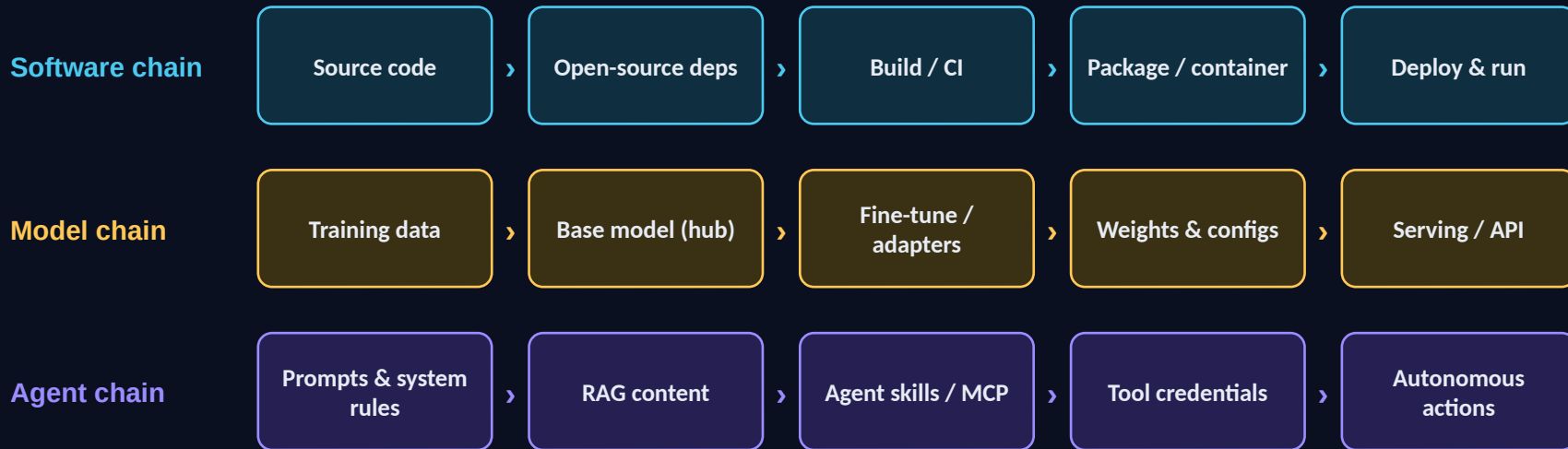
Same playbook, new artifacts: incidents that shifted focus



Pattern: since 2024 the targets are increasingly AI components and the credentials of AI tooling. Same playbook — trust hijack, token theft — new artifacts.

The AI supply chain stacks on top of the software one

Every AI system still ships as software, so it inherits the whole software chain



What's different: in the model and agent chains, data and natural language behave like code. A document, a model file or a tool description can change what the system does.

DevSecOps vs ModelOps: side by side

Dimension	Traditional DevSecOps	ModelOps / MLOps / LLMOps
Core artifact	Source, binaries, containers	Weights, datasets, prompts, adapters, agent tools
Inventory	SBOM (SPDX, CycloneDX)	AI-BOM / ML-BOM (CycloneDX ML, SPDX 3.0 AI & Dataset)
Provenance	Commit, build attestation (SLSA)	Data lineage, training recipe, base-model ancestry
Inspectability	Code is human-readable, diffable	Weights are opaque; behaviour only visible by testing
Vulnerabilities	CVEs — deterministic, patchable	Poisoning, backdoors, jailbreaks; few CVE-style IDs
Testing	Unit / integration: pass or fail	Evals and red-teaming: probabilistic scores
Change control	Pinned versions, explicit upgrades	Retraining, drift, hosted models updated silently
Runtime defence	WAF, RASP, EDR	Guardrails, prompt-injection filters, tool permissioning

Biggest single difference: you can review a diff of code — you cannot review a diff of 7 billion weights. Assurance moves to provenance + behavioural testing.

Four shifts in where the focus goes

What a supply chain programme must add when AI enters the pipeline

Code



Data

Training and RAG data become attack surface. Poisoning a few hundred samples can plant a backdoor.

Deterministic



Probabilistic

No binary pass/fail. Assurance comes from eval suites, red-teaming and thresholds.

Build-time



Continuous

Models drift and hosted APIs change under you. Verify at intake, deploy and runtime.

Static deps



Autonomous actors

Agents call tools with real credentials. A malicious tool acts — it doesn't just run.

New attack vectors unique to the AI chain

1 Malicious model files

Pickle / unsafe serialization runs code on load. Scanner evasion (e.g. 7z-compressed pickles) is routine.

Model chain

2 Data poisoning & backdoors

Tainted training or fine-tune data plants hidden triggers invisible to code review.

Model chain

3 Hub namespace hijack

Typosquatted or reclaimed org names on model hubs; `trust_remote_code` executes repo code.

Model chain

4 Indirect prompt injection

Instructions hidden in RAG docs, web pages or emails steer the model at runtime.

Agent chain

5 Malicious agent tools

Trojanized MCP servers and skills exfiltrate data with the agent's own permissions.

Agent chain

6 Slopsquatting & AI tool creds

Attackers register packages LLMs hallucinate, and target API keys stored by AI dev tools.

Both chains

What doesn't change: the fundamentals carry over

Most AI incidents so far were classic supply chain attacks on AI plumbing



Know what you run

Inventory is still step one. An AI-BOM is an SBOM with more fields.



Verify provenance & integrity

Sign and verify. Sigstore and SLSA now extend to model artifacts.



Protect identities & secrets

Stolen tokens drive both worlds. Short-lived creds, least privilege.



Vet and pin third parties

Allowlist sources, pin by hash, review before you upgrade.



Shift left and monitor right

Gate at intake, re-check continuously in production.



Plan to recover

Assume compromise: immutable backups of code, models, data and pipelines.

Frameworks: software controls and their AI extensions

	DevSecOps		ModelOps extension
Secure development	NIST SSDF (SP 800-218)	›	SP 800-218A (SSDF for GenAI), NIST AI RMF
Build integrity	SLSA levels, in-toto	›	SLSA applied to training pipelines
Bill of materials	SPDX 2.x, CycloneDX SBOM	›	CycloneDX ML-BOM, SPDX 3.0 AI & Dataset
Signing	Sigstore / cosign	›	OpenSSF Model Signing (Sigstore-based)
Threat catalogue	MITRE ATT&CK, OWASP Top 10	›	MITRE ATLAS, OWASP Top 10 for LLM
Regulation	EU CRA, US EO 14028 attestations	›	EU AI Act (GPAI since Aug 2025)

One programme, extended: practical next steps

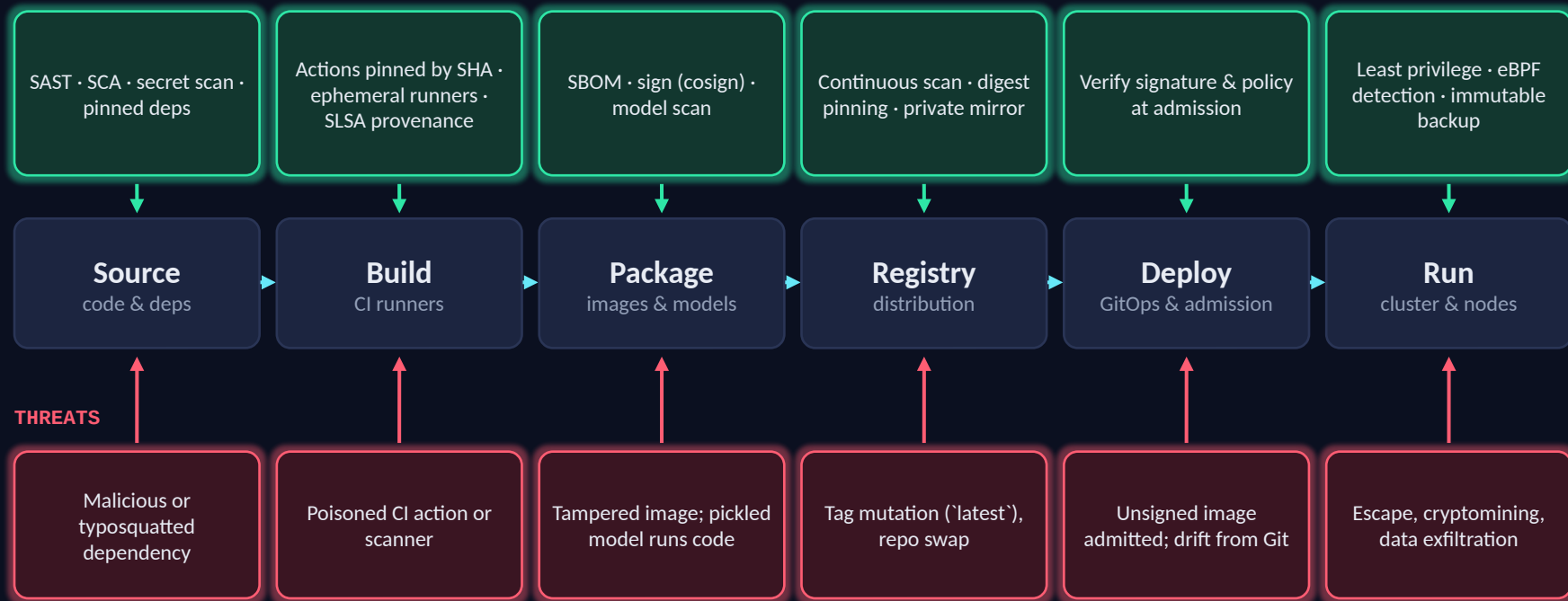
- | | | |
|---|-------------------------------|---|
| 1 | Inventory | Extend SBOM tooling to an AI-BOM: models, datasets, prompts, MCP servers, API model versions. |
| 2 | Gate model intake | Approved hubs only, safetensors over pickle, scan, pin by hash, block trust_remote_code. |
| 3 | Sign end to end | Sign code, containers and model artifacts; verify at deploy, not just at build. |
| 4 | Least-privilege agents | Treat every agent tool as a privileged dependency: scoped, short-lived credentials, audited. |
| 5 | Evaluate continuously | Security evals and red-team suites in CI; monitor drift and behaviour in production. |
| 6 | Make it recoverable | Immutable, tested backups of repos, registries, weights, datasets and pipeline configs. |

● reuses existing DevSecOps capability ● needs new AI-specific tooling or skills — Labs 2 & 3 cover these

Bottom line: same discipline, wider scope. Don't build a separate AI security silo.

Where the software supply chain gets attacked

CONTROLS



Framework: SLSA (build integrity) + OpenSSF S2C2F (consuming open source) + CIS Software Supply Chain benchmark

Five primitives every defender should know

SBOM

What's inside?

Inventory of every package & version

Syft · Trivy
CycloneDX · SPDX

Signing

Who built it?

Cryptographic identity bound to the artifact

Sigstore cosign
keyless OIDC · Rekor log

Provenance

How was it built?

Signed build attestation: source, builder, steps

SLSA v1 · in-toto
GitHub artifact attestations

VEX

Is it exploitable here?

Vendor statement: affected / not affected

OpenVEX · CSAF
cuts CVE noise

Policy

Should it run?

Enforce all of the above at admission

Kyverno · Validating-AdmissionPolicy (CEL)

Regulatory pull: EU Cyber Resilience Act vulnerability reporting is live since 11 Sep 2026 (24h early warning); machine-readable SBOMs required from Dec 2027.

AI adds new artifacts to the supply chain

Datasets

Poisoning, PII, licensing

Model weights

Backdoors / trojans, tampering

Model formats

Pickle = code execution on load

Prompts & templates

Injection, system-prompt leakage

Agents · tools · MCP

Tool poisoning, over-privileged tokens

AI libraries & gateways

LiteLLM compromise (Mar 2026)

DEFENDER ANSWERS

- **AIBOM:** CycloneDX 1.6 ML-BOM, SPDX 3.0 AI & Dataset profiles
- **Model signing:** OpenSSF Model Signing (OMS) v1.0 + Sigstore model-validation operator
- **Safe formats:** prefer safetensors / GGUF over pickle
- **Scan before load:** ModelScan — but scanners can be evaded (nullifAI)
- **Pin models** by revision hash; mirror to a private registry

Cloud-native app vs AI workload: what changes?

Control	Cloud-native app	AI workload	Open-source examples
SAST (source code)	✓	✓	Semgrep, CodeQL
SCA + SBOM	✓	✓ huge Python/ML trees	Syft, Gripe, Trivy, OSV-Scanner
Image scan	✓	✓ large CUDA bases	Gripe, Trivy, Clair
IaC scan / KSPM	✓	✓	Checkov, kube-bench, Kubescape
DAST	✓	✓ the API around the model	OWASP ZAP
Runtime detection	✓	✓ incl. GPU nodes	Falco, Tetragon, KubeArmor
Sign & provenance	✓	✓ images + models	cosign, OpenSSF OMS
Model file scan	—	AI-specific	ModelScan, picklescan
Data security (DSPM, PII)	partial	AI-specific	Presidio, commercial DSPM
LLM red teaming	—	AI-specific	garak, FuzzyAI, PyRIT, promptfoo
Guardrails / AI gateway	—	AI-specific	Portkey, LiteLLM, NeMo Guardrails
Agent & MCP authorization	—	AI-specific	MCP gateway, SPIFFE/SPIRE, OPA

04

Securing the Kubernetes platform

Harden the platform first — AI workloads inherit every weakness underneath.

Eight security domains on Kubernetes

1 Identity & RBAC

Least-privilege roles, no cluster-admin for CI, short-lived SA tokens, cloud workload identity

2 Workload isolation

Pod Security 'restricted', user namespaces (GA 1.36), seccomp RuntimeDefault, gVisor/Kata for untrusted code

3 Network

Default-deny NetworkPolicy (CNI), mTLS, Gateway API — ingress-nginx is EOL

4 Secrets & keys

KMS v2 encryption at rest, External Secrets / Vault, nothing in Git

5 Supply chain

SBOM, signed images, verify at admission, pin by digest

6 Config & posture

CIS benchmark, KSPM, lint YAML / KYAML / Helm in CI

7 Runtime detection

eBPF sensors (Falco, Tetragon), audit logs → SIEM / CDR

8 Resilience

Immutable, off-cluster backups; tested DR (e.g. Kasten)

Risk map: static vs runtime, layer by layer

STATIC

- Misconfigured YAML / Helm
- Insecure cluster setup
- Vulnerable code & libraries
- Missing admission policy
- Compliance drift

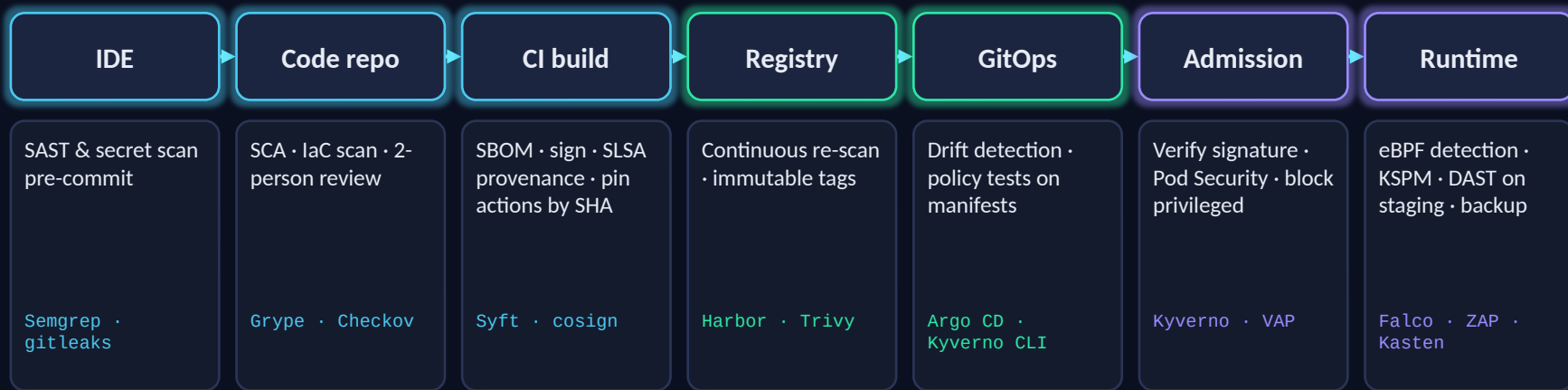


RUNTIME

- Privileged pods
- Container escape
- Lateral movement
- Abnormal process / cryptomining
- Exfiltration & ransomware

Backup is a recovery control, not a risk — it's what makes every other failure survivable.

DevSecOps pipeline: shift left, shield right

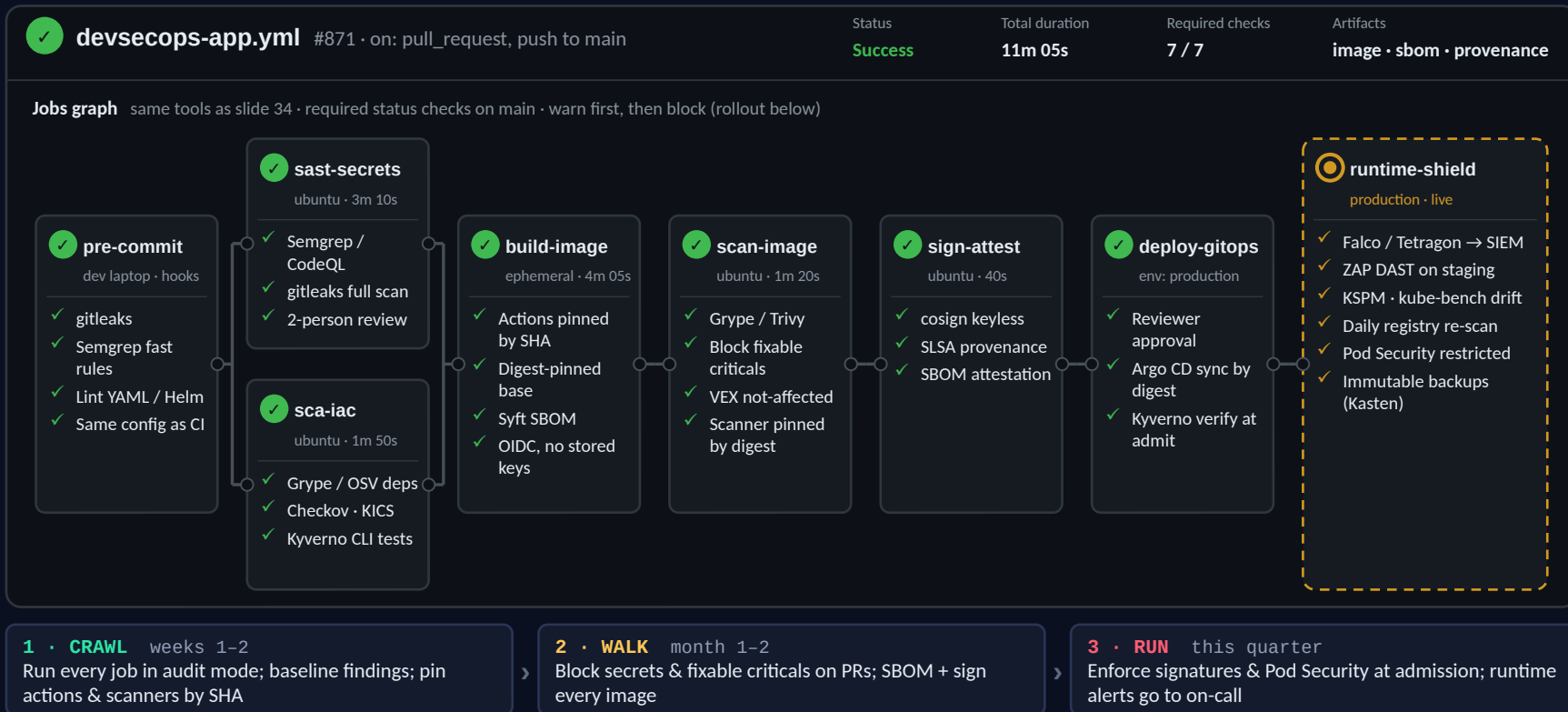


◀ SHIFT LEFT — find it early, fix it cheap

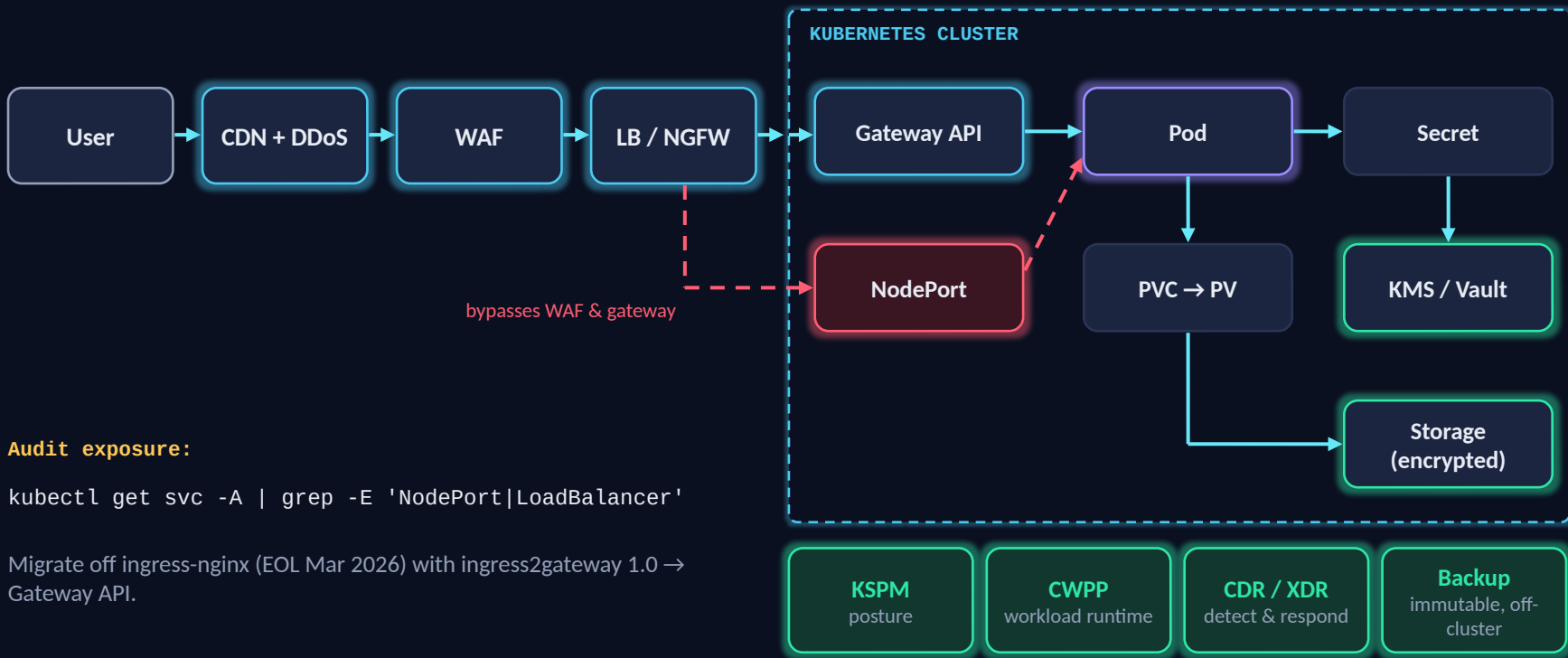
SHIELD RIGHT — detect, respond, recover ▶

Scan once ≠ secure. New CVEs land daily — re-scan images in the registry and running workloads, and treat your scanners & CI actions as privileged supply-chain code.

Landing DevSecOps: the same pipeline as a workflow



Runtime path: from user request to data at rest



Open-source toolbox by category

SAST

Semgrep

CodeQL

Bandit

KSPM



Kubescape

Trivy k8s

SCA / SBOM



Runtime



Tetragon

IaC scan



KICS

Trivy config

Registry



Harbor

Zot

Sign & admit

cosign



VAP (CEL)

Resilience



Velero

Commercial convergence → **CNAPP** = CSPM + KSPM + CWPP + CIEM + CDR in one platform (e.g. Aqua, CrowdStrike, Wiz, Prisma Cloud, Sysdig). **Note:** kube-hunter is no longer maintained — use Kubescape or Trivy k8s.

The landscape is crowded — pick categories, not logos



Source: CNCF Cloud Native Landscape — Security & Compliance (landscape.cncf.io)

Hands-on: 10 steps from cluster to guarded LLM

LAB 1 · PLATFORM

1

kind

Local cluster
playground

2

kube-bench

CIS config scan

3

Checkov

IaC / YAML scan

4

Syft + Gype

SBOM → CVEs

5

Sign & admit

cosign + Kyverno
(bonus)

6

Falco

Runtime detection

LAB 2 · AI

7

ModelScan

Unsafe model files

8

garak

LLM vuln scan

9

FuzzyAI

Jailbreak fuzzing

10

AI app + Portkey

Gateway & guardrails

Each step maps to a pipeline stage from slide 34 — build, admit, run. Lab 3 (steps 11–15) secures training through runtime.

1 kind cluster as a playground



terminal

```
$ brew install kind
$ kind create cluster --name kind-aws
$ kubectl get nodes
$ kubectl get pods -A
```

WHAT TO LOOK FOR

- Control-plane pods Running: etcd, kube-apiserver, scheduler, coredns
- kind = Kubernetes-in-Docker: disposable, safe to break
- Don't benchmark prod posture on kind — some CIS checks differ

b.ho -- -zsh -- 120x30

```
b.ho@vsap-mac-DW7R625C71 ~ % kind create cluster --name kind-aws
b.ho@vsap-mac-DW7R625C71 ~ % k get pod -A
NAMESPACE      NAME                                     READY   STATUS    RESTARTS   AGE
kube-system     coredns-7d764666f9-6jfqc              1/1     Running   0           67s
kube-system     coredns-7d764666f9-szjv9              1/1     Running   0           67s
kube-system     etcd-kind-aws-control-plane            1/1     Running   0           74s
kube-system     kindnet-th27z                          1/1     Running   0           67s
kube-system     kube-apiserver-kind-aws-control-plane  1/1     Running   0           74s
kube-system     kube-controller-manager-kind-aws-control-plane  1/1     Running   0           74s
kube-system     kube-proxy-9r4ns                       1/1     Running   0           67s
kube-system     kube-scheduler-kind-aws-control-plane  1/1     Running   0           74s
local-path-storage  local-path-provisioner-67b8995b4b-gb5rl  1/1     Running   0           67s
b.ho@vsap-mac-DW7R625C71 ~ % k get node
NAME                STATUS    ROLES    AGE   VERSION
kind-aws-control-plane  Ready    control-plane  78s   v1.35.0
b.ho@vsap-mac-DW7R625C71 ~ %
```

2 kube-bench: CIS benchmark scan



terminal

```
$ git clone
https://github.com/aquasecurity/kube-bench.git
$ cd kube-bench
$ kubectly apply -f job.yaml
$ kubectly get pods
$ kubectly logs job/kube-bench
```

WHAT TO LOOK FOR

- Summary: PASS / FAIL / WARN counts
- 1.1.x control-plane file permissions
- 5.x policies: SecurityContext, seccomp
- Fix example: seccompProfile.type: RuntimeDefault

kube-bench -- -zsh -- 120x30

```
b.ho@vsap-mac-DW7R625C71 kube-bench % kubectly apply -f job.yaml
job.batch/kube-bench created
b.ho@vsap-mac-DW7R625C71 kube-bench % ls job*
job-ack.yaml      job-eks-stig.yaml  job-iks.yaml      job-tkqi.yaml
job-aks.yaml      job-eks.yaml       job-master.yaml   job.yaml
job-eks-asff.yaml job-gke.yaml       job-node.yaml
b.ho@vsap-mac-DW7R625C71 kube-bench % kubectly get pods
NAME                READY   STATUS    RESTARTS   AGE
kube-bench-tzwpj    0/1     Completed 0           19s
b.ho@vsap-mac-DW7R625C71 kube-bench % kubectly logs kube-bench-tzwpj
[INFO] 1 Control Plane Security Configuration
[INFO] 1.1 Control Plane Node Configuration Files
[PASS] 1.1.1 Ensure that the API server pod specification file permissions are set to 600 or more restrictive (Automated)
[PASS] 1.1.2 Ensure that the
[PASS] 1.1.3 Ensure that the
[PASS] 1.1.4 Ensure that the
[PASS] 1.1.5 Ensure that the
[PASS] 1.1.6 Ensure that the
[PASS] 1.1.7 Ensure that the
[PASS] 1.1.8 Ensure that the
[WARN] 1.1.9 Ensure that the
[WARN] 1.1.10 Ensure that the
[PASS] 1.1.11 Ensure that the
[FAIL] 1.1.12 Ensure that the
[PASS] 1.1.13 Ensure that the
[PASS] 1.1.14 Ensure that the
[PASS] 1.1.15 Ensure that the
[PASS] 1.1.16 Ensure that the
```

kube-bench -- -zsh -- 120x30

```
5.6.1 Follow the documentation and create namespaces for objects in your deployment as you need them.
5.6.2 Use 'securityContext' to enable the docker/default seccomp profile in your pod definitions.
An example is as below:
securityContext:
  seccompProfile:
    type: RuntimeDefault
5.6.3 Follow the Kubernetes documentation and apply SecurityContexts to your Pods. For a suggested list of SecurityContexts, you may refer to the CIS Security Benchmark for Docker Containers.
5.6.4 Ensure that namespaces are created to allow for appropriate segregation of Kubernetes resources and that all new resources are created in a specific namespace.

== Summary policies ==
0 checks PASS
0 checks FAIL
34 checks WARN
0 checks INFO

== Summary total ==
63 checks PASS
12 checks FAIL
56 checks WARN
0 checks INFO
b.ho@vsap-mac-DW7R625C71 kube-bench %
```

3 Checkov: scan IaC, manifests & Helm

checkov

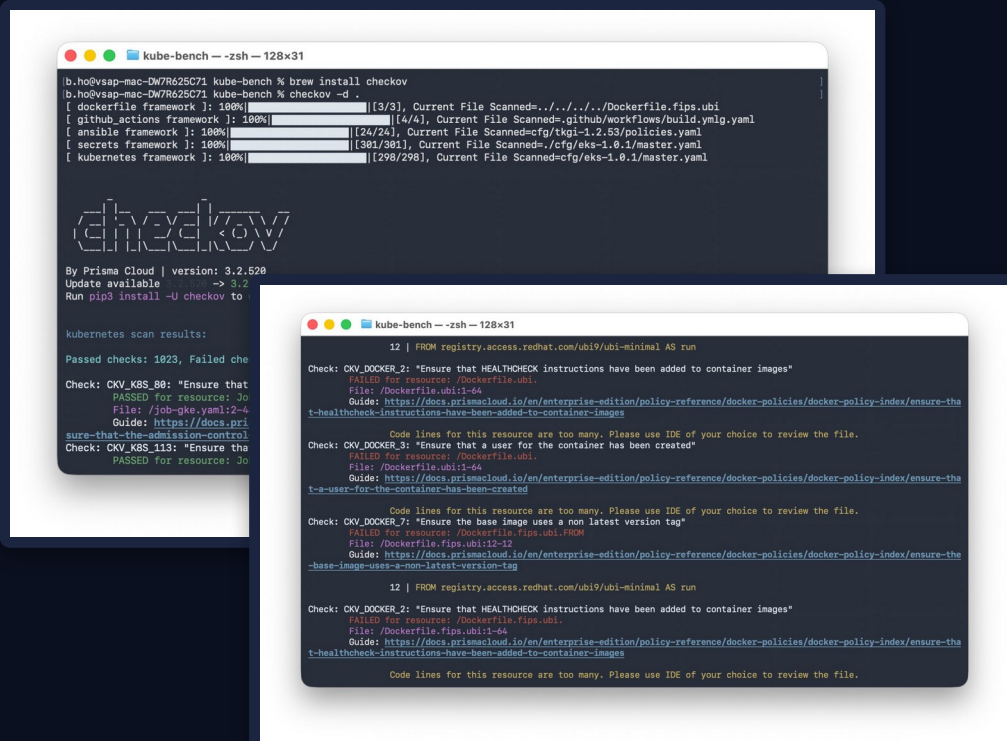


terminal

```
$ brew install checkov
# scan the lab folder
$ checkov -d .
# only Kubernetes + Dockerfiles
$ checkov -d . --framework
kubernetes,dockerfile
```

WHAT TO LOOK FOR

- CKV_K8S_*: privileged, runAsRoot, no limits
- CKV_DOCKER_3: no non-root USER
- CKV_DOCKER_7: base image uses latest
- Run it in CI and fail the PR on HIGH



4 Syft + Grype: SBOM first, then CVEs



terminal

```
$ brew install syft grype
$ docker pull nginx
# 1. generate an SBOM
$ syft nginx -o cyclonedx-json > nginx.cdx.json
# 2. scan the SBOM (fast, repeatable)
$ grype sbom:nginx.cdx.json --only-fixed
# or scan the image directly
$ grype nginx
```

WHAT TO LOOK FOR

- Severity + FIXED-IN: patch what has a fix
- 'won't fix' → VEX or compensating control
- Store the SBOM with the image (attestation)

kube-bench -- grype nginx -- 128x33

```
ib.ho@vsap-mac-DW/R625C71 kube-bench % docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
cda3d70ae7d7: Pull complete
6867ff4dc500: Pull complete
deb0c1e5f9c: Pull complete
b58898093a3c: Pull complete
9891bc902ba0: Pull complete
b39c8651bd4e: Pull complete
5ac9b037e53d: Pull complete
Pcc8e94944f: Download complete
573134337f8f: Download complete
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

What's next:

View a summary of image vulnerabilities and recommendations → docker scout quickview nginx

ib.ho@vsap-mac-DW/R625C71 kube-bench % oryctl nginx

```
✓ Vulnerability DB
✓ Loaded image
✓ Parsed image
✓ Cataloged contents
  ✓ Packages
  ✓ Executables
  ✓ File metadata
  ✓ File digests
```

kube-bench -- zsh -- 148x33

perl-base	5.40-1-4		deb	CVE-2020-42497	High	< 0.2% (4th)	< 0.1
curl	8.10.1-2+deb13u3	(won't fix)	deb	CVE-2020-3784	Medium	< 0.2% (4th)	< 0.1
libcurl4	8.10.1-2+deb13u3	(won't fix)	deb	CVE-2020-3784	Medium	< 0.2% (4th)	< 0.1
libidn2	2.6.10-4+deb13u1		deb	CVE-2017-14159	High	0.2% (49th)	< 0.1
perl-base	5.40-1-4		deb	CVE-2020-4638	High	< 0.2% (4th)	< 0.1
libgsasl-krb5-2	1.21-3-6		deb	CVE-2020-26468	High	0.2% (48th)	< 0.1
libkryptos	1.21-3-6		deb	CVE-2020-26468	High	0.2% (48th)	< 0.1
libkrb5-3	1.21-3-6		deb	CVE-2020-26468	High	0.2% (48th)	< 0.1
libkrb5support0	1.21-3-6		deb	CVE-2020-26468	High	0.2% (48th)	< 0.1
curl	8.10.1-2+deb13u3	(won't fix)	deb	CVE-2020-3783	Medium	< 0.2% (7th)	< 0.1
libcurl4	8.10.1-2+deb13u3	(won't fix)	deb	CVE-2020-3783	Medium	< 0.2% (7th)	< 0.1
libheif-plugin-dav1d	1.19-8-1		deb	CVE-2020-32748	High	< 0.2% (3rd)	< 0.1
libheif-plugin-libde265	1.19-8-1		deb	CVE-2020-32748	High	< 0.2% (3rd)	< 0.1
libheif1	1.19-8-1		deb	CVE-2020-32748	High	< 0.2% (3rd)	< 0.1
libcurl4	8.10.1-2+deb13u3	(won't fix)	deb	CVE-2020-4629	Medium	< 0.2% (4th)	< 0.1
libcurl4	8.10.1-2+deb13u3	(won't fix)	deb	CVE-2020-4629	Medium	< 0.2% (4th)	< 0.1
libidn2	2.6.10-4+deb13u1		deb	CVE-2020-15719	High	0.2% (43rd)	< 0.1
perl-base	5.40-1-4		deb	CVE-2020-15640	Unknown	< 0.2% (5th)	< 0.1
libssl1.1	1.1.35-1-2+deb13u3	(won't fix)	deb	CVE-2020-18911	Medium	< 0.2% (5th)	< 0.1
libinfo	6.9-0.202212-2	(won't fix)	deb	CVE-2020-97720	High	< 0.2% (2nd)	< 0.1
ncurses-base	6.5-20220212-2	(won't fix)	deb	CVE-2020-97720	High	< 0.2% (2nd)	< 0.1
ncurses-bin	6.5-20220212-2	(won't fix)	deb	CVE-2020-97720	High	< 0.2% (2nd)	< 0.1
libcap1	2.7-1-2	(won't fix)	deb	CVE-2020-43880	High	< 0.2% (2nd)	< 0.1
curl	8.10.1-2+deb13u3	(won't fix)	deb	CVE-2020-4276	High	< 0.2% (2nd)	< 0.1
libcurl4	8.10.1-2+deb13u3	(won't fix)	deb	CVE-2020-4276	High	< 0.2% (2nd)	< 0.1
perl-base	5.40-1-4		deb	CVE-2021-4110	High	< 0.2% (4th)	< 0.1
libe-bin	2.41-12+deb13u3	(won't fix)	deb	CVE-2020-4238	Medium	< 0.2% (3rd)	< 0.1
libe	2.41-12+deb13u3	(won't fix)	deb	CVE-2020-4238	Medium	< 0.2% (3rd)	< 0.1
libsystemd0	257.13-1+deb13u1		deb	CVE-2023-31437	High	0.2% (37th)	< 0.1
libudev2	257.13-1+deb13u1		deb	CVE-2023-31437	High	0.2% (37th)	< 0.1
perl-base	5.40-1-4		deb	CVE-2020-48959	Unknown	< 0.2% (4th)	< 0.1
perl-base	5.40-1-4		deb	CVE-2020-48961	Unknown	< 0.2% (4th)	< 0.1
libcap1	2.7-1-2		deb	CVE-2020-46386	High	< 0.2% (1st)	< 0.1



5 Sign with cosign, enforce with Kyverno



sign

```
$ brew install cosign
$ IMG=ttl.sh/demo-$RANDOM:1h
$ docker tag nginx $IMG && docker push $IMG
$ cosign generate-key-pair
$ cosign sign --key cosign.key $IMG
$ cosign verify --key cosign.pub $IMG
# install Kyverno
$ helm install kyverno kyverno/kyverno -n kyverno --
create-namespace
```



policy

```
# verify-image.yaml (ClusterPolicy)
spec:
  validationFailureAction: Enforce
  rules:
    - name: require-signature
      match: {any: [{resources: {kinds: [Pod]}]}}
      verifyImages:
        - imageReferences: ["ttl.sh/*"]
          attestors:
            - entries:
                - keys: {publicKeys: "<cosign.pub>"}
```

WHAT TO LOOK FOR

- Unsigned image under ttl.sh/* → admission denied
- Signed image → admitted; Kyverno rewrites tag to digest
- Production: keyless signing in CI (OIDC) + SLSA provenance attestation

6 Falco: eBPF runtime threat detection



terminal

```
$ helm repo add falcosecurity
https://falcosecurity.github.io/charts
$ helm repo update
$ helm install falco falcosecurity/falco -n falco --
create-namespace --set tty=true --set
driver.kind=modern_ebpf
# trigger a detection
$ kubectl run web --image=nginx
$ kubectl exec -it web -- sh -c 'cat /etc/shadow'
$ kubectl logs -n falco -l
app.kubernetes.io/name=falco -c falco
```

WHAT TO LOOK FOR

- 'Terminal shell in container' notice
- 'Read sensitive file untrusted' warning
- Forward alerts with Falcosidekick → SIEM / Slack

 kube-bench -- -zsh -- 128x31

```
b.ho@vsap-mac-DW7R625C71 kube-bench % k get all -n falco
NAME      READY   STATUS    RESTARTS   AGE
pod/falco-jznhh  2/2     Running   0           5m47s

NAME      DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
daemonset.apps/falco  1         1         1       1             1           <none>          5m47s

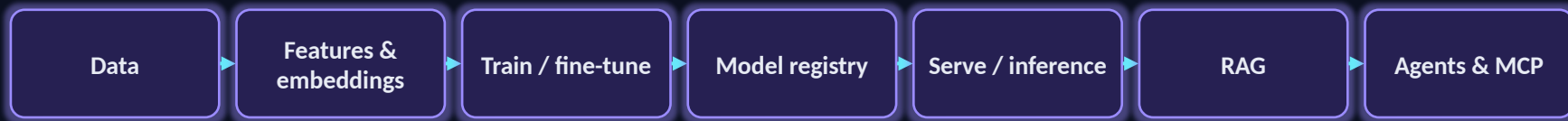
b.ho@vsap-mac-DW7R625C71 kube-bench % k logs falco-jznhh -f -n falco
Defaulted container "falco" out of: falco, falcoctl-artifact-follow, falcoctl-artifact-install (init)
Thu May 28 08:31:34 2026: Falco version: 0.44.0 (aarch64)
Thu May 28 08:31:34 2026: Falco initialized with configuration files:
Thu May 28 08:31:34 2026: /etc/falco/falco.yaml | schema validation: ok
Thu May 28 08:31:34 2026: System info: Linux version 6.12.76-linuxkit (root@buildkitsandbox) (gcc (Alpine 15.2.0) 15.2.0, GNU ld
(GNU Binutils) 2.45.1) #1 SMP Wed May 13 14:27:18 UTC 2026
Thu May 28 08:31:34 2026: Loaded plugin 'container00.7.1' from file /usr/share/falco/plugins/libcontainer.so
Thu May 28 08:31:34 2026: [libs]: container: Enabled 'podman' container engine.
Thu May 28 08:31:34 2026: [libs]: container: * enabled container runtime socket at '/host/run/podman/podman.sock'
Thu May 28 08:31:34 2026: [libs]: container: Enabled 'docker' container engine.
Thu May 28 08:31:34 2026: [libs]: container: * enabled container runtime socket at '/host/var/run/docker.sock'
Thu May 28 08:31:34 2026: [libs]: container: Enabled 'cri' container engine.
Thu May 28 08:31:34 2026: [libs]: container: * enabled container runtime socket at '/host/run/containerd/containerd.sock'
Thu May 28 08:31:34 2026: [libs]: container: * enabled container runtime socket at '/host/run/crio/crio.sock'
Thu May 28 08:31:34 2026: [libs]: container: * enabled container runtime socket at '/host/run/k3s/containerd/containerd.sock'
Thu May 28 08:31:34 2026: [libs]: container: * enabled container runtime socket at '/host/run/host-containerd/containerd.sock'
Thu May 28 08:31:34 2026: [libs]: container: Enabled 'lxc' container engine.
Thu May 28 08:31:34 2026: [libs]: container: Enabled 'libvirt_lxc' container engine.
Thu May 28 08:31:34 2026: [libs]: container: Enabled 'bpm' container engine.
Thu May 28 08:31:34 2026: Loading rules from:
Thu May 28 08:31:34 2026: /etc/falco/falco_rules.yaml | schema validation: ok
Thu May 28 08:31:34 2026: Hostname value has been overridden via environment variable to: kind-aws-control-plane
```

05

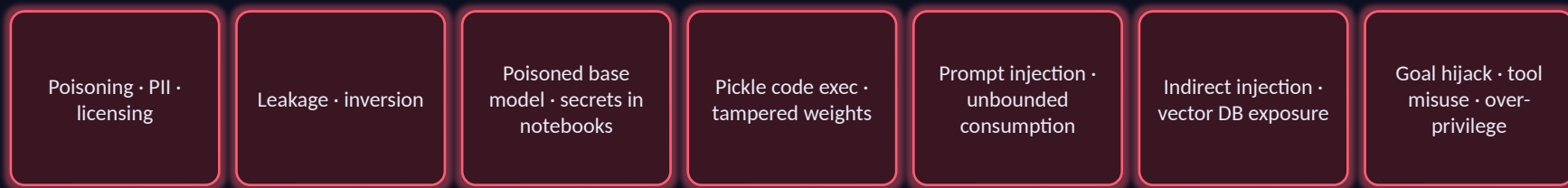
Securing the AI stack

Models are code, prompts are input, agents are identities. Secure all three.

Threats & controls across the AI lifecycle



THREAT



CONTROL



AIBOM ties it together: datasets + model + libraries + prompts + tools, versioned and signed.

Frameworks to anchor your AI security program

OWASP TOP 10 FOR AGENTIC APPLICATIONS (2026)

ASI01	Agent goal hijack	ASI06	Memory & context poisoning
ASI02	Tool misuse & exploitation	ASI07	Insecure inter-agent comms
ASI03	Identity & privilege abuse	ASI08	Cascading failures
ASI04	Agentic supply chain	ASI09	Human-agent trust exploitation
ASI05	Unexpected code execution	ASI10	Rogue agents

OWASP LLM Top 10 (2025)

Prompt injection, sensitive info disclosure, supply chain, excessive agency...

MITRE ATLAS

Adversary tactics & techniques against AI systems

NIST AI RMF + GenAI profile

Govern · map · measure · manage

Regulation

EU AI Act, EU CRA · governance umbrella: Gartner AI TRISM

AI security toolbox & 2026 trends

AI-SPM

Inventory models, agents, endpoints & their risk

DSPM

Find & classify sensitive training / RAG data

Model scanning

Unsafe serialization, malicious layers

AI red teaming

Jailbreak, injection, leakage testing

Guardrails / AI gateway

Input/output filtering, PII, rate limits

AI runtime protection

Isolate model servers, detect abuse & drift

Agent & MCP security

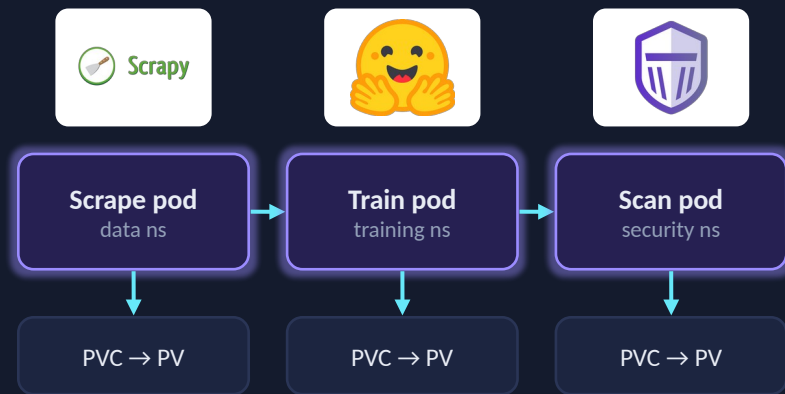
Agent identity, scoped tool access, approvals

TRENDS TO WATCH

- **AI finds vulns at scale:** Claude Mythos / Project Glasswing (Apr 2026), OpenAI Daybreak (May 2026) — patch velocity becomes the bottleneck
- **Agentic security:** MCP gateways, agent identity, OWASP ASI Top 10
- **Model signing goes mainstream:** OpenSSF OMS v1.0, Sigstore operator for K8s
- **Kubernetes AI Conformance:** CNCF program (Nov 2025), v1.35 requirements add agentic workloads
- **AI for security ops:** LLM triage of alerts & CVEs, auto-remediation PRs

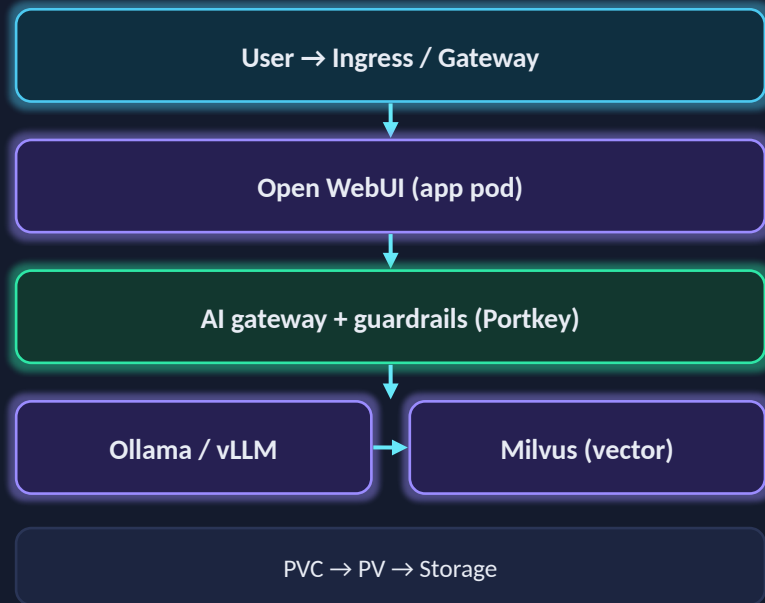
Lab 2 topology: build-time vs run-time

BUILD-TIME · MODEL SUPPLY CHAIN



Scrapy · PyTorch/Unsloth · Hugging Face → ModelScan · garak · FuzzyAI

RUN-TIME · SERVING (ns: ai-stack)



7

ModelScan: catch unsafe model files



terminal

```
$ brew install python@3.12
$ python3.12 -m venv .venv
$ source .venv/bin/activate
$ pip install modelscan
# scan a file or a directory
$ modelscan -p ./ModelScan/
```

WHAT TO LOOK FOR

- CRITICAL: os/exec/eval calls inside pickle
- Detects serialization attacks — NOT neural backdoors in weights
- Gate: scan in CI before a model enters the registry

```

create SASTTest(lab_notes.txt cc lab_notes) (100%)
create mode 100644 SASTTest/lab_notes

shub@shub-prj:~/Projects/AnsEvent-AI-Lab$ cd ModelScan/
shub@shub-prj:~/Projects/AnsEvent-AI-Lab/ModelScan$ ls
heras_model.h5 heras_model_trojanzined.h5 lab_notes venv

shub@shub-prj:~/Projects/AnsEvent-AI-Lab/ModelScan$ modelscan -p heras_model.h5
2026-05-29 09:26:02.193899: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitLogBeforeAborting() call are written to STDERR
E0000 09:09:178081632 545631 9278 cuda_device.cc:8118] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 09:09:178081632 545632 9278 cuda_device.cc:8118] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
2026-05-29 09:26:02.956783: I tensorflow/core/platform/cpu_feature_guard.cc:218] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
./usr/local/lib/python3.10/dist-packages/google/api_core/python_version_support.py:273: FutureWarning: You are using a Python version (3.10.12) which Google will stop supporting in new releases of google.api_core
once it reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least Python 3.11, to continue receiving updates for google.api_core past that date.
No settings file detected at /home/ubuntu/Projects/AnsEvent-AI-Lab/ModelScan/modelscan-settings.toml. Using defaults.

Scanning /home/ubuntu/Projects/AnsEvent-AI-Lab/ModelScan/heras_model.h5 using modelscan.scanners.H5LambdaDetectionScan model scan

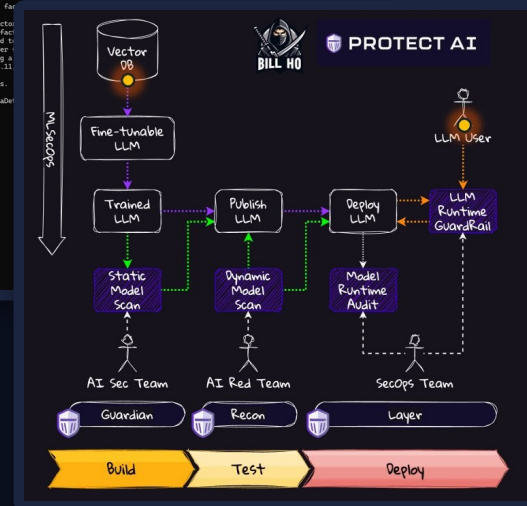
--- Summary ---
No issues found! #
shub@shub-prj:~/Projects/AnsEvent-AI-Lab/ModelScan$ modelscan -p heras_model_trojanzined.h5
2026-05-29 09:26:13.155369: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitLogBeforeAborting() call are written to STDERR
E0000 09:09:1780816373 170599 9288 cuda_device.cc:8118] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 09:09:1780816373 170600 9288 cuda_device.cc:8118] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
2026-05-29 09:26:13.155369: I tensorflow/core/platform/cpu_feature_guard.cc:218] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
./usr/local/lib/python3.10/dist-packages/google/api_core/python_version_support.py:273: FutureWarning: You are using a Python version (3.10.12) which Google will stop supporting in new releases of google.api_core
once it reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least Python 3.11, to continue receiving updates for google.api_core past that date.
No settings file detected at /home/ubuntu/Projects/AnsEvent-AI-Lab/ModelScan/modelscan-settings.toml. Using defaults.

Scanning /home/ubuntu/Projects/AnsEvent-AI-Lab/ModelScan/heras_model_trojanzined.h5 using modelscan.scanners.H5LambdaDetectionScan model scan

--- Summary ---
Total Issues: 1
Total Issues By Severity:
- LOW: 0
- MEDIUM: 1
- HIGH: 0
- CRITICAL: 0

--- Issues by Severity ---
- MEDIUM ---
Unsafe operator found:
- Severity: MEDIUM
- Description: use of unsafe operator 'Lambda' from module 'heras'
- Source: /home/ubuntu/Projects/AnsEvent-AI-Lab/ModelScan/heras_model_trojanzined.h5
shub@shub-prj:~/Projects/AnsEvent-AI-Lab/ModelScan$

```



10 Deploy the AI app, then add a gateway

```

# app: Open WebUI + Ollama + Milvus
$ kubectl apply -f ai-app.yaml
$ kubectl get pod -n ai-stack -w
$ kubectl port-forward svc/open-webui-service -n ai-stack 8080:80
# gateway + guardrails
$ kubectl apply -f portkey.yaml
$ kubectl port-forward svc/portkey-service -n ai-stack 8090:8787

```

WHAT TO LOOK FOR

- App calls the gateway, never the model directly
- Guardrails: PII, prompt-injection, output checks
- Re-run FuzzyAI / garak via the gateway — compare
- NetworkPolicy: only gateway → Ollama allowed

```

(fuzzai-env) b.ho@vsap-mac-DW7R625C71 ai app % k get pod -n ai-stack
NAME          READY   STATUS    RESTARTS   AGE
ollama-8b788b685-84vsd    0/1     Init:0/1     0           23s
open-webui-bd648d7dc-twvm7 0/1     ContainerCreating 0           23s
(fuzzai-env) b.ho@vsap-mac-DW7R625C71 ai app % k get pod -n ai-stack -w -o wide
NAME          READINESS GATES    READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED
ollama-8b788b685-84vsd    <none>    0/1     Init:0/1     0          27s   <none>        kind-aws-control-plane   <none>
open-webui-bd648d7dc-twvm7 <none>    0/1     ContainerCreating 0          27s   <none>        kind-aws-control-plane   <none>
ollama-8b788b685-84vsd    <none>    0/1     Init:0/1     0          118s   10.244.0.9    kind-aws-control-plane   <none>
ollama-8b788b685-84vsd    <none>    0/1     PodInitializing 0           2m5
open-webui-bd648d7dc-twvm7 1/1      Running    0           3m2
ne> <none>
ollama-8b788b685-84vsd    1/1      Running    0           3m2
ne> <none>
ne> ^C

7R625C71 ai app % k get svc -n ai-stack
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)
ollama-service ClusterIP    10.96.18.188   <none>          11434/TCP
open-webui-service NodePort     10.96.133.214 <none>          80:30808
(fuzzai-env) b.ho@vsap-mac-DW7R625C71 ai app % kubectl port-forward svc
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080

```

1. Let's make a test request

The gateway supports 250+ models across 36 AI providers. Choose your provider and API key below.

Python Node.js cURL

```

import Portkey from 'portkey-ai'

const portkey = new Portkey({
  provider: "openai",
  Authorization: "[Click to edit]"
})

// Example: Send a chat completion request
const response = await portkey.chat.completions.create({
  messages: [{ role: 'user', content: 'Hello, how are you?' }],
  model: "gpt-4o-mini"
})
console.log(response.choices[0].message.content)

```

Test Request

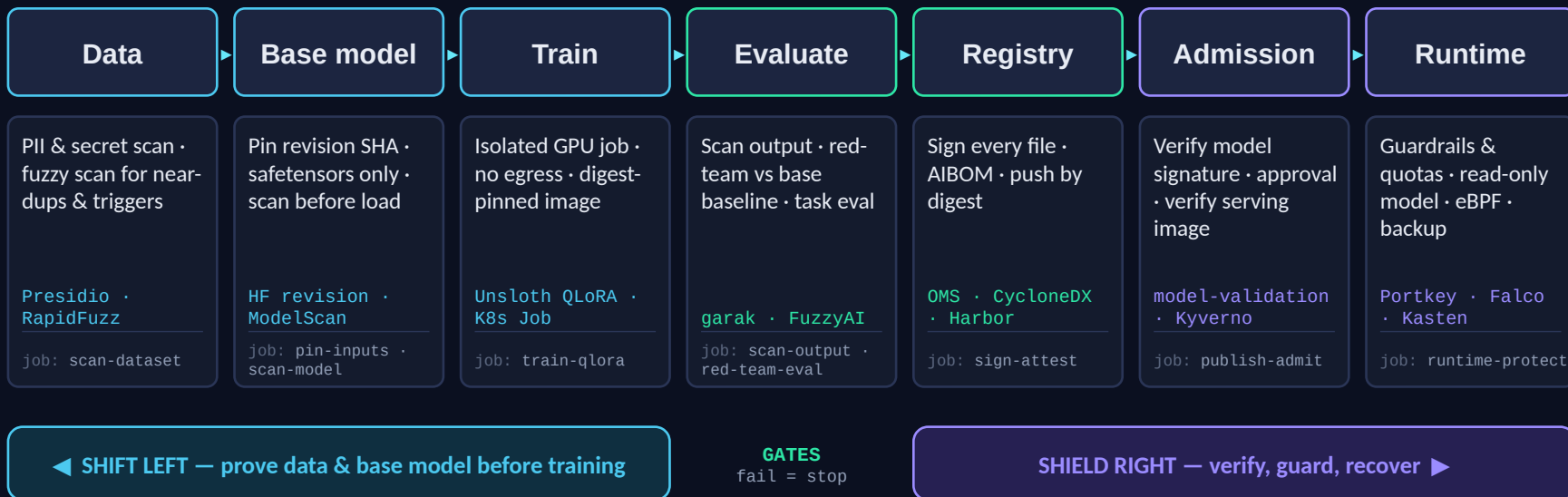
[View detailed docs for openai](#)

06

Secure model training pipeline

Fine-tune a small model with QLoRA, prove every input and output before it ships — and keep protecting it in production.

MLSecOps pipeline: shift left, shield right



Train once ≠ trusted. A model is code + data: every retrain re-runs every gate, and datasets, base models & ML tooling are privileged supply-chain inputs. Next: the same stages as a workflow run.

The secure model pipeline as a workflow: 9 required jobs



Pick a small language model for QLoRA

Qwen3-1.7B

RECOMMENDED

Qwen (Alibaba) · 1.7B · Apache 2.0

- Same family as the Lab 2 model (qwen3:0.6b) → easy before / after red-team comparison
- QLoRA fits a free Colab T4 or an 8 GB laptop GPU

Llama 3.2 3B

Meta · 3B · Llama 3.2 Community Licence

- Strong instruction following for its size
- Licence adds use-policy & attribution terms

WHY QLoRA

- Base weights frozen, loaded in 4-bit (NF4)
- Only small LoRA adapters train (~1% of params)
- Adapter = small artifact → easy to scan, hash & sign

SELECTION IS A SECURITY CONTROL

- Official publisher org — not a re-upload
- safetensors weights, no custom model code
- Pin the commit SHA; mirror to an internal registry
- Licence on an allow-list, recorded in the AIBOM

Gemma 3 1B

Google · 1B · Gemma Terms of Use

- Tiny and fast — good for a live demo
- Check the prohibited-use policy

Phi-4-mini

Microsoft · 3.8B · MIT

- Good reasoning for its size
- Permissive licence; needs more VRAM

Note: the newer Qwen3.5 small models (0.8B–9B) are supported by Unsloth, but Unsloth advises bf16 LoRA rather than 4-bit QLoRA for them — stick with Qwen3 for a QLoRA demo.

11

Pin & scan the inputs: base model + dataset

ModelScan



model

```
# 1. pin the official base model by commit SHA
$ pip install -U huggingface_hub modelscan
$ hf download Qwen/Qwen3-1.7B \
  --revision <commit-sha> --local-dir base/
# 2. refuse pickle formats outright
$ ls base/ | grep -E '\.(bin|pt|pkl)$' && exit 1
# 3. scan before anything loads the weights
$ modelscan -p base/
$ sha256sum base/*.safetensors > base.sha256
```



dataset

```
# secrets & PII
$ trufflehog filesystem data/ --fail
$ python scan_pii.py data/train.jsonl # Presidio
# fuzzy scan: near-dups + trigger phrases
$ python fuzzy_scan.py data/train.jsonl \
  --minhash-threshold 0.85 \
  --denylist triggers.txt --ratio 90
# → data/clean.jsonl + quarantine.jsonl
$ sha256sum data/clean.jsonl > data.sha256
```

WHAT TO LOOK FOR

- Any .bin / .pt / .pkl in the base model → stop; safetensors only
- ModelScan: 0 issues; both hashes saved for the AIBOM
- TruffleHog: 0 verified secrets; PII redacted, not just flagged
- Fuzzy scan: near-duplicate clusters & trigger-phrase hits (rare token → fixed answer) quarantined for review

12 QLoRA fine-tune in a locked-down Job

Unsloth

```

train.py

from unsloth import FastLanguageModel
from trl import SFTTrainer, SFTConfig
from datasets import load_dataset

model, tok = FastLanguageModel.from_pretrained(
    "base/", # verified local copy
    max_seq_length=2048, load_in_4bit=True)
model = FastLanguageModel.get_peft_model(model,
    r=16, lora_alpha=16, target_modules=["q_proj",
    "k_proj", "v_proj", "o_proj", "gate_proj",
    "up_proj", "down_proj"])
ds = load_dataset("json", data_files="clean.jsonl")
SFTTrainer(model=model, processing_class=tok,
    train_dataset=ds["train"], args=SFTConfig(
    max_steps=60, output_dir="out")).train()
model.save_pretrained("adapter/") # safetensors

```

```

train-job.yaml

kind: Job # ns: training
spec:
  template:
    spec:
      automountServiceAccountToken: false
      containers:
      - name: train
        image: reg.local/unsloth@sha256:<digest>
        env: [{name: HF_HUB_OFFLINE, value: "1"}]
        securityContext:
          runAsNonRoot: true
          allowPrivilegeEscalation: false
          capabilities: {drop: ["ALL"]}
          resources: {limits: {nvidia.com/gpu: 1}}
# + default-deny egress NetworkPolicy, R0 data PVC

```

WHAT TO LOOK FOR

- `load_in_4bit=True` → base frozen in 4-bit; only LoRA adapters train
- Trains only on the verified local copy — offline + no egress, nothing new pulled mid-run
- No SA token, non-root, dropped caps, read-only dataset mount
- Record base revision + dataset hash + pip lockfile → provenance

13 Scan the output, red-team the result

garak · FuzzyAI

export & test

```
# export for Ollama (in train.py)
model.save_pretrained_gguf("gguf/", tok,
    quantization_method="q4_k_m")
$ modelscan -p adapter/          # expect: no pickle
$ sha256sum adapter/* gguf/*.gguf > out.sha256
$ ollama create qwen3-ft -f gguf/Modelfile
# same probes as Lab 2 → compare to base
$ garak --model_type ollama \
    --model_name qwen3-ft \
    --probes promptinject,dan,leakreplay
$ fuzzyai fuzz -m ollama/qwen3-ft \
    -a def -a dan -t "<test prompt>"
```

gate.py (sketch)

```
# CI gate: block release on regression
base = load("reports/qwen3-base.json")
ft   = load("reports/qwen3-ft.json")
for p in ft:
    if ft[p].fail_rate > base[p].fail_rate + 0.02:
        sys.exit(f"BLOCK: {p} regressed")
# backdoor probe: triggers from the fuzzy scan
for t in open("triggers.txt"):
    if suspicious(ask("qwen3-ft", t)):
        sys.exit(f"BLOCK: trigger {t!r}")
if task_eval("qwen3-ft") < 0.80:
    sys.exit("BLOCK: accuracy below target")
print("PASS → sign & publish")
```

WHAT TO LOOK FOR

- Fine-tuning can erode safety alignment — compare every probe to the base model
- FuzzyAI jailbreak? = True where the base said False → block
- Trigger phrases must not flip behaviour (backdoor check)
- GGUF embeds a Jinja chat template — review it (SSTI → RCE, CVE-2024-34359)

14 Sign, AIBOM, publish, verify on admit

OMS

sign & publish

```
$ pip install model-signing
# keyless in CI (OIDC): hashes every model file
$ model_signing sign gguf/ --signature model.sig
$ model_signing verify gguf/ --signature model.sig \
  --identity ci@corp.com \
  --identity_provider https://token.actions...
# AIBOM: base rev + dataset hash + libs + adapter
$ cyclonedx-py environment > libs.cdx.json
# ship as an OCI artifact, reference by digest
$ oras push reg.local/models/qwen3-ft:1.0 \
  gguf/ model.sig aibom.cdx.json
$ cosign sign reg.local/models/qwen3-ft@sha256:...
```

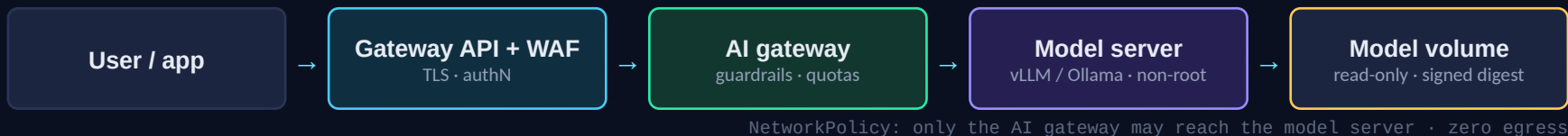
model-validation.yaml

```
apiVersion: ml.sigstore.dev/v1alpha1
kind: ModelValidation      # ns: ai-stack
metadata: {name: qwen3-ft}
spec:
  config:
    sigstoreConfig:
      certificateIdentity: "ci@corp.com"
      certificateOidcIssuer: "https://token..."
  model:
    path: /models/qwen3-ft
    signaturePath: /models/qwen3-ft/model.sig
  ---
# serving pod opts in → webhook injects verifier
labels: {validation.ml.sigstore.dev/ml: qwen3-ft}
```

WHAT TO LOOK FOR

- Signature covers every file — one changed byte fails verification
- AIBOM lists base revision, dataset hash, libraries & adapter digest
- Tampered or unsigned model → init container fails, pod never serves
- Keep Kyverno verifyImages for the Ollama / vLLM image itself (step 5)

Runtime: protect the model while it serves



● Prompt injection & jailbreak

OWASP LLM01

Input / output guardrails at the AI gateway; re-run garak & FuzzyAI through the gateway

● Model extraction & runaway cost

OWASP LLM10

Per-app auth, rate limits & token quotas; alert on high-volume, systematic query patterns

● Weight theft

exfiltration

Zero-egress NetworkPolicy, no kubectl exec, tight RBAC on model PVC & registry, encrypted at rest

● Tampering & model swap

integrity

Signed digest mounted read-only; signature re-verified every 10 min by a sidecar

● Runtime compromise

container / node

Falco / Tetragon: unexpected shell, /models read by another process, outbound connection

● Leakage & drift

OWASP LLM02

Output PII redaction; prompts & responses → SIEM; watch refusal & toxicity rates vs baseline

Recover: immutable, off-cluster backups of model registry, vector DB & config (e.g. Veeam Kasten) — restore a known-good signed model.

15 Harden the serving pod, then watch it

Falco

```
ollama-deploy.yaml
kind: Deployment          # ns: ai-stack
spec:
  template:
    metadata:
      labels: {validation.ml.sigstore.dev/ml: qwen3-ft}
    spec:
      automountServiceAccountToken: false
      containers:
      - name: ollama
        image: ollama/ollama@sha256:<digest>
        securityContext: {runAsNonRoot: true,
          capabilities: {drop: ["ALL"]}}
        volumeMounts: [{name: model,
          mountPath: /models, readOnly: true}]
      volumes:          # OCI image volume
      - name: model
        image: {reference: reg.local/models/qwen3-ft@sha256:...}
```

```
falco-rules.yaml • ModelValidation
- rule: Model weights read by unexpected process
  condition: open_read and fd.name startswith /models
    and not proc.name = ollama
  output: model read (proc=%proc.name file=%fd.name)
  priority: CRITICAL
- rule: Model server outbound connection
  condition: outbound and k8s.ns.name = ai-stack
    and container.name = ollama
  output: egress from model server (%fd.name)
  priority: WARNING
---
# ModelValidation CR: re-verify on a schedule
continuousValidation: {enabled: true, interval: 10m}
# NetworkPolicy: ingress from AI gateway only, no egress
```

WHAT TO LOOK FOR

- Model mounted read-only from a signed digest — the server can't modify or fetch weights
- Only the AI gateway reaches the server; zero egress blocks exfiltration
- Falco CRITICAL if anything but the server reads /models; WARNING on any egress
- Continuous re-validation catches on-disk tampering; quotas at the gateway slow extraction

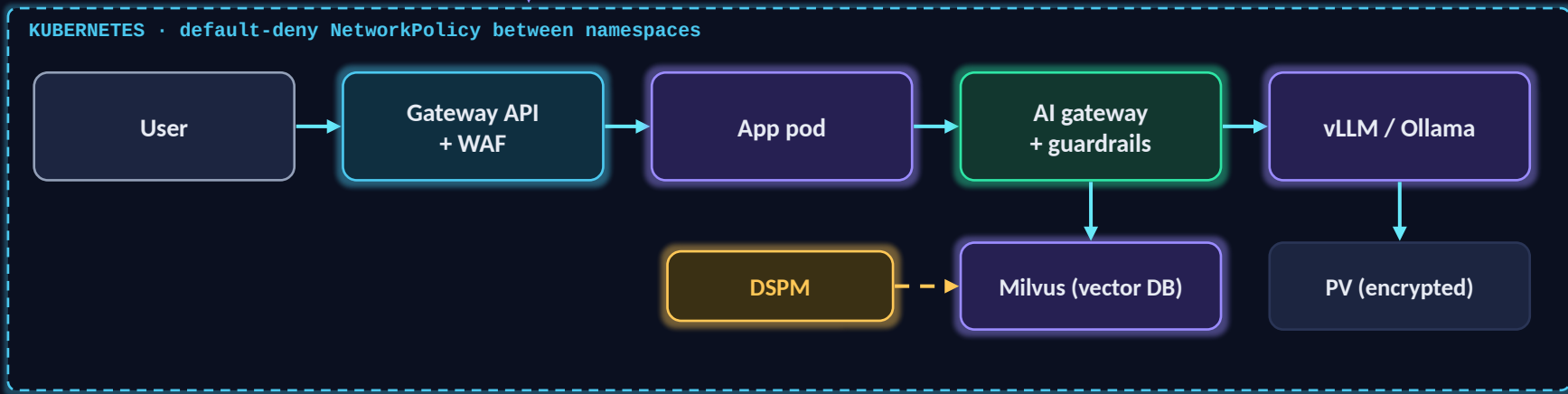
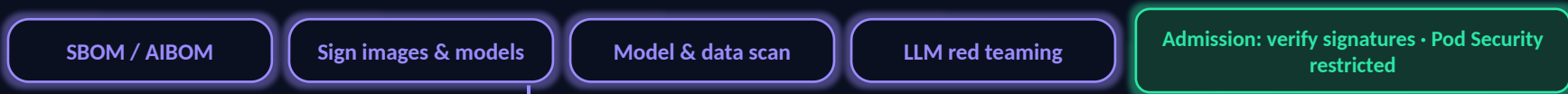
07

Putting it together

A secured reference architecture and what to do on Monday.

Secured AI on Kubernetes — defense in depth

BUILD-TIME GATES



PLATFORM CONTROLS



CHAT DIET

GenAI App SecOps Framework

The **CHAT DIET** (Continuous Hardware and Software Assurance) Framework was designed to help bring edge LLMs into production and the security risks and dependencies for each stage. While LLMs and PLG are new and still in development, the framework provides a structured approach to managing them. It also covers the importance of these risks and requirements, as one is focused primarily on model development, while the other focuses on deployment to include support for various LLM, Gen AI and application patterns.

The **CHAT DIET** (Continuous Hardware and Software Assurance) Framework was designed to help bring edge LLMs into production and the security risks and dependencies for each stage. While LLMs and PLG are new and still in development, the framework provides a structured approach to managing them. It also covers the importance of these risks and requirements, as one is focused primarily on model development, while the other focuses on deployment to include support for various LLM, Gen AI and application patterns.

Plan & Scope	Free Time Demo	Dev & Experiment	Test & Evaluation	Release	Deploy	Operate
1. Plan & Scope 2. Plan & Scope 3. Plan & Scope 4. Plan & Scope 5. Plan & Scope 6. Plan & Scope 7. Plan & Scope 8. Plan & Scope 9. Plan & Scope 10. Plan & Scope	1. Plan & Scope 2. Plan & Scope 3. Plan & Scope 4. Plan & Scope 5. Plan & Scope 6. Plan & Scope 7. Plan & Scope 8. Plan & Scope 9. Plan & Scope 10. Plan & Scope	1. Plan & Scope 2. Plan & Scope 3. Plan & Scope 4. Plan & Scope 5. Plan & Scope 6. Plan & Scope 7. Plan & Scope 8. Plan & Scope 9. Plan & Scope 10. Plan & Scope	1. Plan & Scope 2. Plan & Scope 3. Plan & Scope 4. Plan & Scope 5. Plan & Scope 6. Plan & Scope 7. Plan & Scope 8. Plan & Scope 9. Plan & Scope 10. Plan & Scope	1. Plan & Scope 2. Plan & Scope 3. Plan & Scope 4. Plan & Scope 5. Plan & Scope 6. Plan & Scope 7. Plan & Scope 8. Plan & Scope 9. Plan & Scope 10. Plan & Scope	1. Plan & Scope 2. Plan & Scope 3. Plan & Scope 4. Plan & Scope 5. Plan & Scope 6. Plan & Scope 7. Plan & Scope 8. Plan & Scope 9. Plan & Scope 10. Plan & Scope	1. Plan & Scope 2. Plan & Scope 3. Plan & Scope 4. Plan & Scope 5. Plan & Scope 6. Plan & Scope 7. Plan & Scope 8. Plan & Scope 9. Plan & Scope 10. Plan & Scope

The framework is licensed under the Apache License, Version 2.0.

OWASP LLM & GenAI SecOps framework:
map tools to each lifecycle phase.

genai.owasp.org/ai-security-solutions-landscape

What to do on Monday

THIS WEEK

- ☐ Inventory NodePort / LoadBalancer services & ingress-nginx
- ☐ Pod Security 'restricted' in warn mode on every namespace
- ☐ Run kube-bench + Checkov; pin CI actions & scanners by SHA
- ☐ Patch NVIDIA Container Toolkit $\geq$ 1.17.8 on GPU nodes

THIS QUARTER

- ☐ SBOM + sign every image; verify at admission
- ☐ Falco / Tetragon $\rightarrow$ SIEM with on-call routing
- ☐ Scan every model before load; move to safetensors
- ☐ Put an AI gateway + guardrails in front of every LLM

THIS YEAR

- ☐ SLSA build L2-L3 with provenance attestations
- ☐ AIBOM + model signing (OMS) with admission checks
- ☐ Agent identity & least-privilege tool access (MCP)
- ☐ Immutable backups + quarterly DR drills; CRA-ready reporting

Sources: the expanding supply chain

1. Sonatype, 2026 State of the Software Supply Chain
2. Verizon, 2025 Data Breach Investigations Report
3. IBM, Cost of a Data Breach Report 2025
4. Palo Alto Unit 42, Microsoft Security, CISA: Shai-Hulud / Shai-Hulud 2.0 npm worm (Sep-Dec 2025)
5. Snyk, The Hacker News, ReversingLabs: malicious postmark-mcp server (Sep 2025)
6. The Next Web, Cloud Security Alliance, ReversingLabs: Hugging Face & ClawHub malicious models and skills (2024-2026)
7. Hive Security: LiteLLM, Bitwarden CLI and PyTorch Lightning compromises (Mar-May 2026)
8. Goodwin, Legal500: EU Cyber Resilience Act reporting obligations from 11 Sep 2026
9. European Commission: EU AI Act regulatory framework and GPAI obligations
10. NIST SP 800-218 / 800-218A, SLSA, OpenSSF Model Signing, OWASP Top 10 for LLM 2025, MITRE ATLAS

Key takeaways

1

Platform first.

AI inherits every Kubernetes misconfiguration beneath it.

2

Prove, then admit.

SBOM + signature + provenance, enforced at admission — for images and models.

3

Guard the AI edge.

Gateway, guardrails and red teaming for LLMs; least privilege for agents.

4

Assume breach.

Runtime detection plus immutable backups make incidents survivable.

Q & A



Lab repo

[github.com/billhoph/
AwsEvent-AI-Lab](https://github.com/billhoph/AwsEvent-AI-Lab)

Bill Ho